ECE391
Computer System Engineering

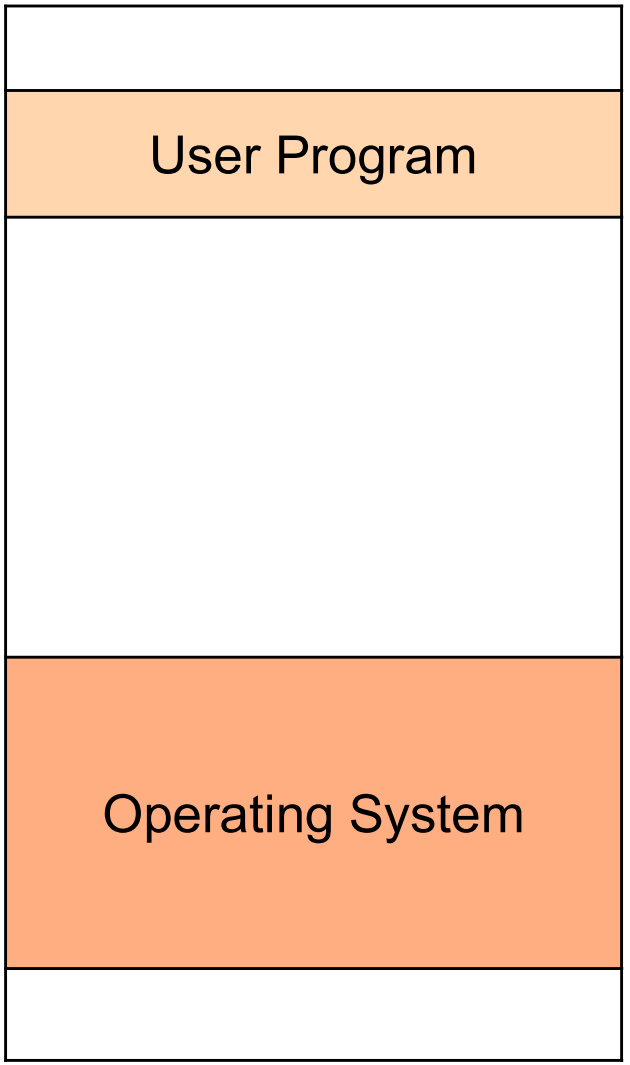
Lecture 17

University of Illinois at Urbana–Champaign

Spring 2024

Programs

- Programs see a simple world
- Loaded into memory at some standard place
- Only thing running on computer
- Interacts with abstracted devices
- *It is our job to maintain this illusion for the programs*



Sharing the CPU

- How can we create the illusion of multiple programs executing simultaneously?
- Each program executes for some time
- The OS periodically:
  - *Suspends* the executing program, and
  - *Resumes* executing another program
- Each executing program is called a *process*
  - Note: In Linux the concept of a *process* is more complex

Context Switch

- How do we suspend and resume a process?
- A process has an execution context (state) consisting of
  - CPU general purpose registers
  - Program counter (instruction pointer)
  - Stack pointer
  - Address space and contents of memory
  - Internal OS state for process (e.g. open files)
- To suspend a process, we need to save all of that state
- To resume, we need to restore all of that state
- This is called a *context switch*

Context Switch?

- We perform something like a context switch when we execute an interrupt handler
  - Save EIP, ESP, and other registers of currently executing program
    - Hardware saves CS, EIP, SS, ESP; handler saves the rest
  - Switch to kernel stack
    - New SS and ESP from TSS (will discuss shortly)
  - Jump to handler code
    - New CS and EIP from interrupt gate
- Executing interrupt handler is not considered a true context switch, although it has many elements of it

Processes and Threads

- A ***thread*** is a basic unit of execution consisting of a stack, program counter, and registers
  - Also called a *lightweight process (LWP)*
  - Also called a *task* in some older OSes
- A traditional Unix process has a single thread
- Modern OSes allow a process to have *multiple* threads
- All threads of the same process share the memory space
- **Process:** address space + threads + OS state (e.g. open files)
  - OS resources and permissions are associated with processes
- **Thread:** stack + PC + SP + registers

Thread Context Switching

- Context switch between threads can be done in user space
  - Do not need privileged execution mode
- Thread context: stack + PC + SP + general purpose registers
- Let's implement threads in user space as an exercise

Process Context Switching

- A process has an execution context (state) consisting of
  - CPU general purpose registers
  - Program counter (instruction pointer)
  - Stack pointer
  - Address space and contents of memory
  - Internal OS state for process (e.g. open files)
- To switch from one process to another processes:
  - Switch executing thread context
  - Update memory map (PDBR)

  } **hardware context**

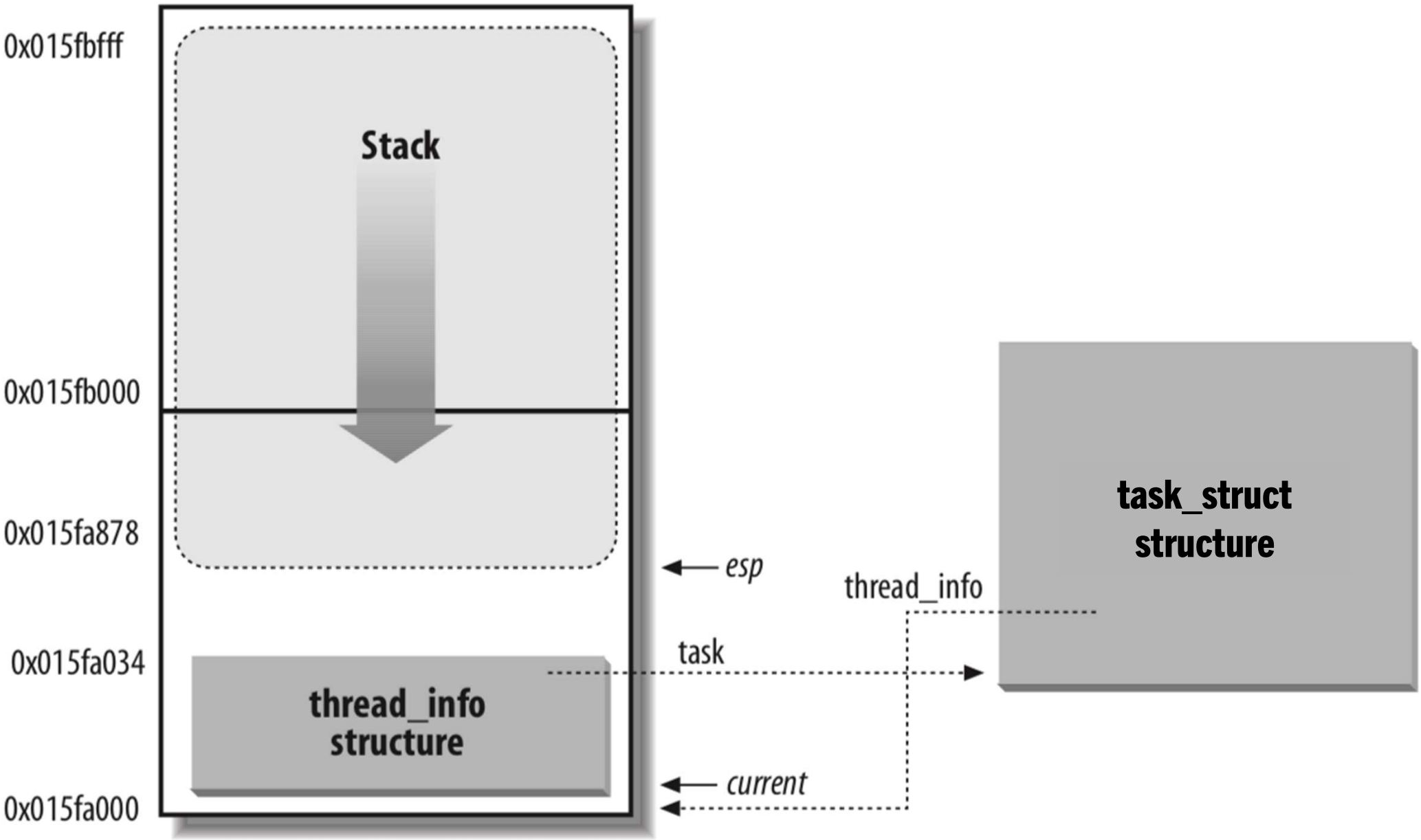
  - Internal OS state (which process is currently executing)

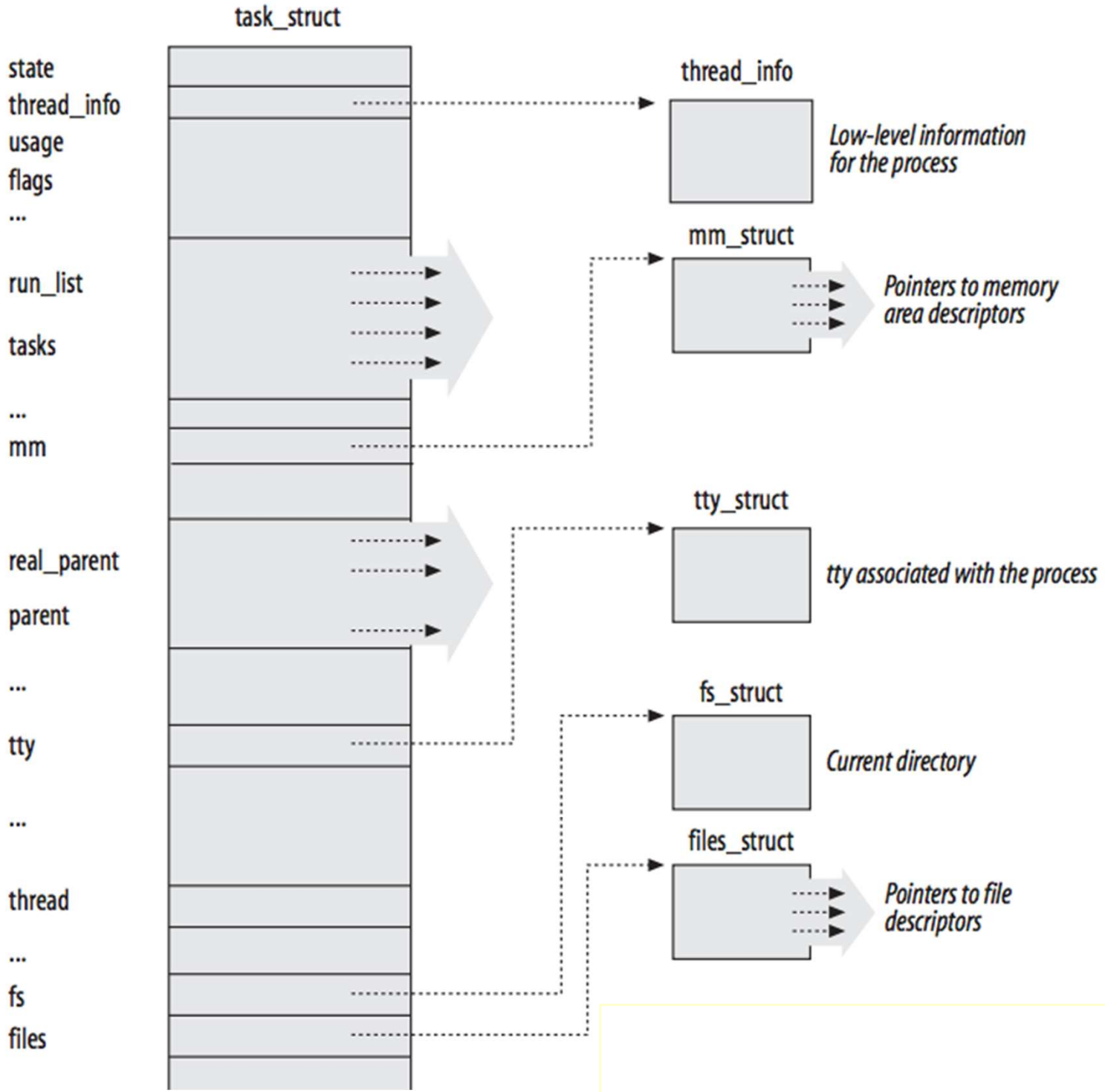
Linux Terminology

- **Original Unix model:** a process is an address space, OS resources, and single execution context
- **Modern Unix model:** a process is an address space, OS resources, and a set of threads
- **Linux model:** lightweight processes that may share address space and OS resources
  - *Thread group* is the equivalent of the Unix notion of process
  - getpid(), kill(), _exit() system calls affect thread group, which is the expected Unix behavior

Linux Process Data Structures

- Two adjacent pages allocated for kernel-mode stack
- thread_info: small structure at the base of stack block
  - Stack grows down from highest address of stack block
- task_struct: main architecture-independent process struct
  - thread_info has pointer to task_struct and vice versa


0x015fbfff
Stack
0x015fb000
0x015fa878
esp
thread_info
task_struct
structure
0x015fa034
task
thread_info
structure
current
0x015fa000



K. Levchenko

Context Switch

- Process context switch happens when in kernel
  - During a system call (e.g. blocking operation)
  - From an interrupt (preëmption)
- **Cooperative multitasking/multithreading:** context switch occurs only when thread explicitly yields control or makes a blocking system call
- **Preëmptive multitasking/multithreading:** context switch may happen at any time
- **Preëmptible kernel:** kernel code may be preëmpted (forced context switch)
  - Linux 2.6+ is a preëmptible kernel

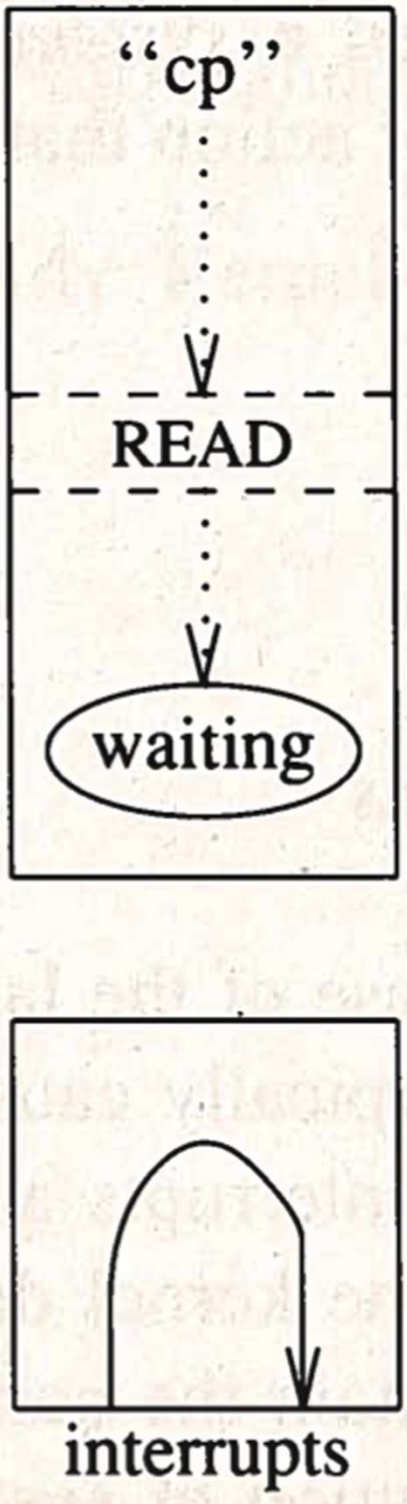
BSD 4.3

User Process

Top Half of Kernel

(not preemptive)

Bottom Half of Kernel



Preemptive scheduling;
cannot block;
runs on user stack in
user address space

Runs until blocked or done;
can block to wait for resource;
runs on kernel stack in
user address space

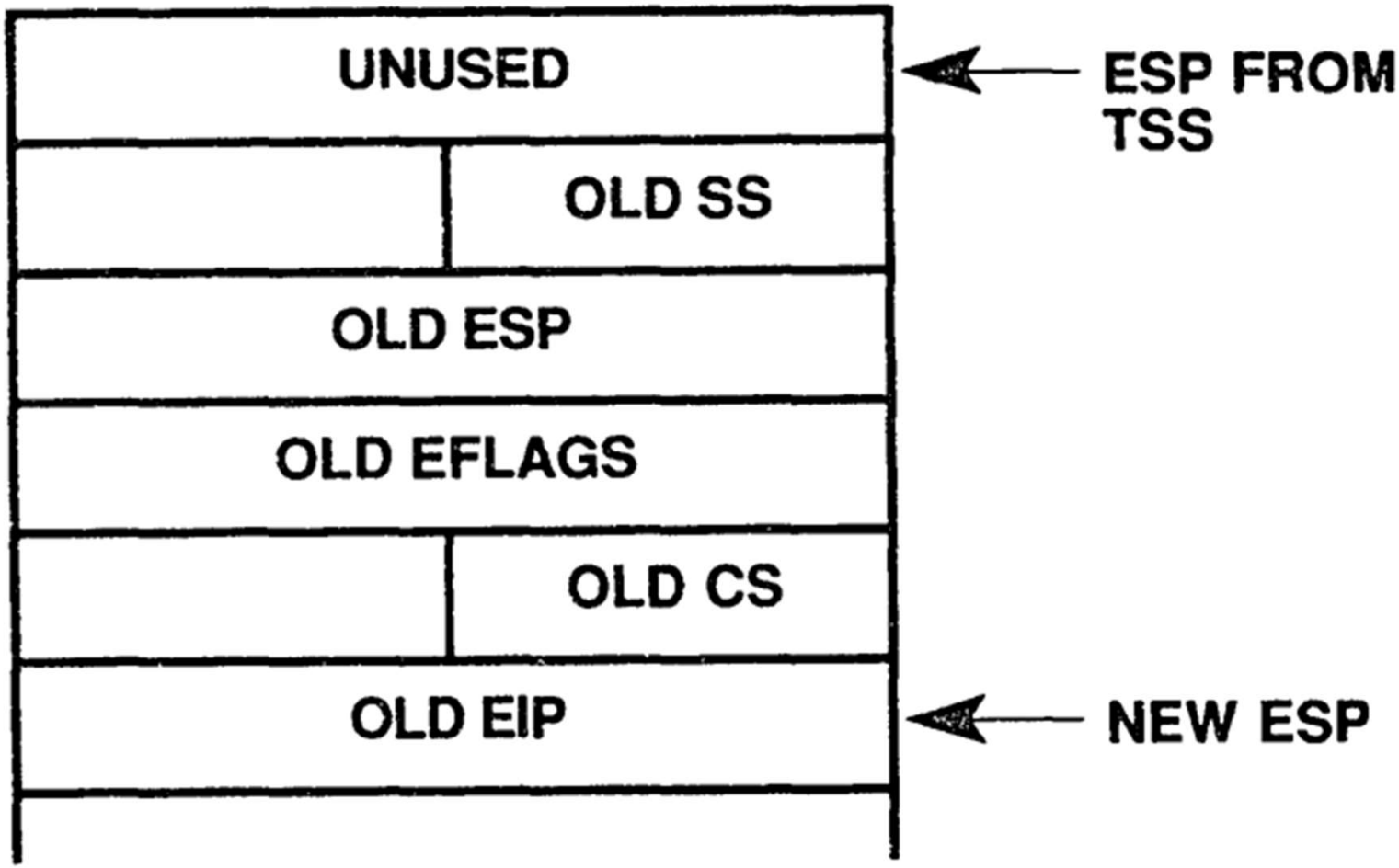
Never scheduled;
cannot block;
runs on kernel stack in
kernel address space

System Call Entry and Exit

```
// user program reads one character from stdin:
//
//     char c;
//     read(0, &c, 1);
//
// read() is declared as:
//
//     ssize_t read(int fd, void *buf, size_t count);
//
// in system call, arguments passed in registers:
//
// %eax ← system call number (3 for read)
// %ebx ← fd (0)
// %ecx ← buf (&c)
// %edx ← count (1)
//

        movl    $3, %eax
        movl    $0, %ebx
        movl    %esp, %ecx # &c
        movl    $1, %edx
        int     $0x80
        ...
```

Stack on Entry to System Call Handler



```
system_call:
        pushl   %gs
        pushl   %fs
        pushl   %es
        pushl   %ds
        pushl   %ebp
        pushl   %esi
        pushl   %edi
        pushl   %edx
        pushl   %ecx
        pushl   %ebx
        pushl   %eax

        // do system call stuff

        popl    %eax
        popl    %ebx
        popl    %ecx
        popl    %edx
        popl    %edi
        popl    %esi
        popl    %ebp
        popl    %ds
        popl    %es
        popl    %fs
        popl    %gs

        iret
```

Blocking Inside the Linux Kernel

- A system call (invoked by the user process) may need to wait for some event (e.g. data to become available)
  1. Inserts self into list of processes waiting for some event
     - Each possible event has an associated wait queue
  2. Sets task state to suspended (`TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE`)
     - These refer to process interruption (signals) *not* interrupts!
  3. Calls `schedule()` to switch to another runnable process
     - Resumes execution when `schedule()` returns

Linux Wait Queues

- A wait queue is a doubly-linked list of waiting processes
- `DECLARE_WAITQUEUE(node, tptr)`
  - Declares an *element* named node of wait queue for task
    - `struct task_struct * tptr`
    - Use `current` for pointer to current task's `task_struct`
  - Note: this is not the declaration of the queue itself
- `DECLARE_WAIT_QUEUE_HEAD(wq)`
  - Declares a wait queue named `wq`
- `add_wait_queue(&wq, &node)`
  - Adds node to wait queue

Linux Work Queue Example

/linux/v2.6.7/source/drivers/char/random.c

```
static DECLARE_WAIT_QUEUE_HEAD(random_read_wait);

static ssize_t random_read(...) {
    DECLARE_WAITQUEUE(wait, current);

    …

    set_current_state(TASK_INTERRUPTIBLE);
    add_wait_queue(&random_read_wait, &wait);

    schedule();

    set_current_state(TASK_RUNNING);
    remove_wait_queue(&random_read_wait, &wait);

    …

}
```

Linux's `schedule()`

- Picks a runnable process to run
  - Which process is chosen depends on scheduler strategy
- Does a lot of other stuff
  - Usage time accounting, process queue (runqueue) updates, *etc.*
- Switches to address space of process being resumed
  - Write CR3 (PDBR) with address of next processes page dir.
- Calls `switch_to()` to perform actual context switch
  - Switches hardware context (ESP, EIP, regs.)
- Execution continues in new process context
- Full user-space context restored on exit from system call

The Current Process in Linux

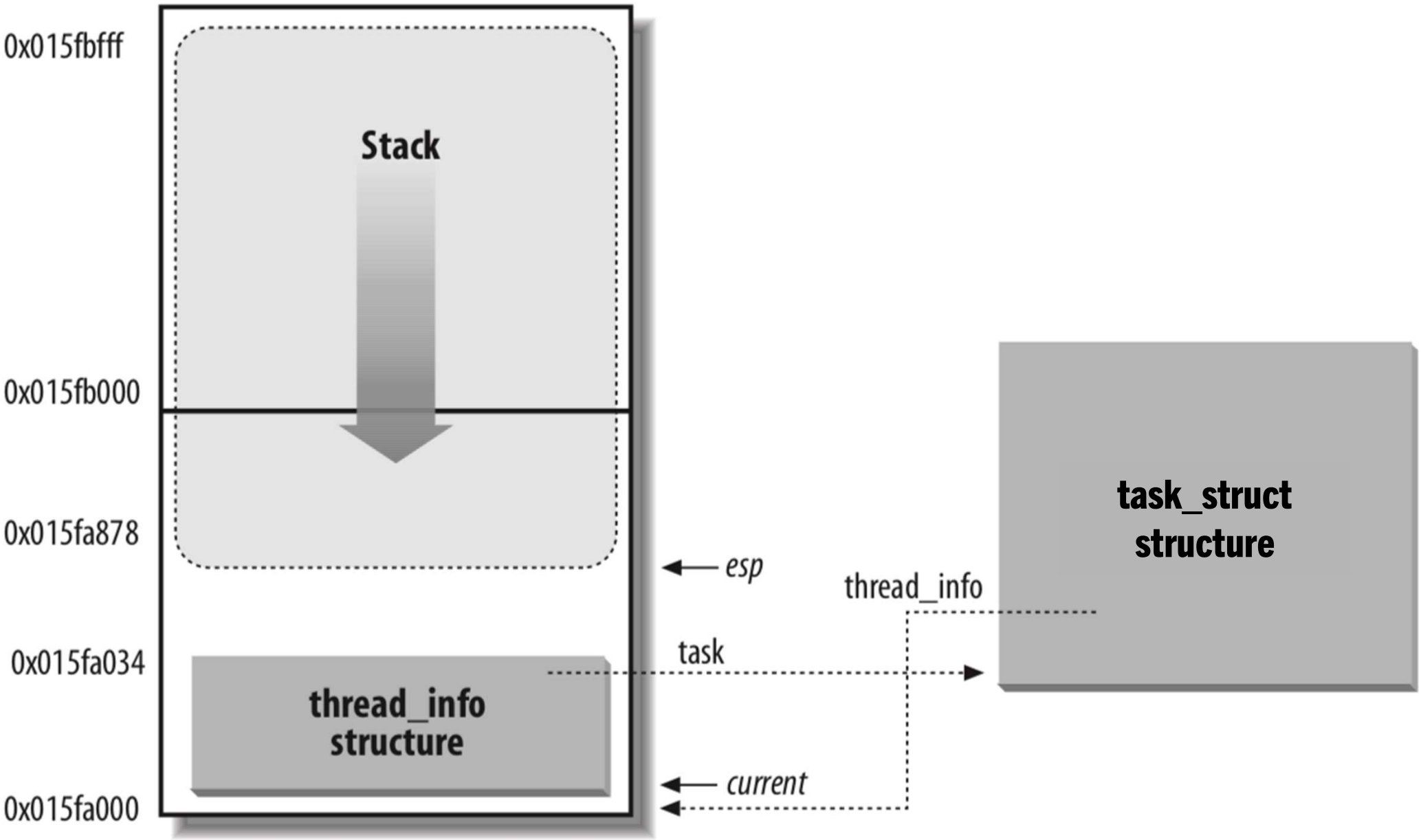
- `current` is a pointer to task_struct of current process
- This is actually a macro:

```
struct task_struct;
static inline struct task_struct * get_current(void) {
    return current_thread_info()->task;
}
#define current get_current()
static inline struct thread_info *current_thread_info(void) {
       struct thread_info *ti;
       __asm__("andl %%esp,%0; ":"=r" (ti) : "0" (~(8172-1)));
       return ti;
}
```

The Current Process in Linux

- `current` is a pointer to task_struct of current process
- This is actually a macro:

```
struct task_struct;
static inline struct task_struct * get_current(void) {
    return current_thread_info()->task;
}
#define current get_current()
static inline struct thread_info *current_thread_info(void) {
       struct thread_info *ti;
       __asm__("andl %%esp, 0xFFFFE000" ::);
       return ti;
}
```


0x015fbfff
Stack
0x015fb000
0x015fa878
esp
thread_info
task_struct
structure
0x015fa034
task
thread_info
structure
current
0x015fa000

The Need for Preemptive Multitasking

- Process context switch can occur in system call
- What if process doesn't make a system call?
- Want to limit process execution time even if it never makes a system call—how?
- Need to get control back to the kernel: *interrupts!*
- Set a periodic timer interrupt, say every 10ms
- Kernel gets control back in timer interrupt handler
- Can we context switch before leaving interrupt handler?
- Why not? Just need to re-enable interrupts

Preemptive Multitasking

- OS sets up a periodic timer interrupt
- Before returning from any interrupt:
  - Re-enable interrupts
  - If needed, call `schedule()`
  - Return to user process
- *What is happening actually?*

```
system_call:
        pushl   %gs
        pushl   %fs
        pushl   %es
        pushl   %ds
        pushl   %ebp
        pushl   %esi
        pushl   %edi
        pushl   %edx
        pushl   %ecx
        pushl   %ebx
        pushl   %eax

        // do system call stuff

        popl    %eax
        popl    %ebx
        popl    %ecx
        popl    %edx
        popl    %edi
        popl    %esi
        popl    %ebp
        popl    %ds
        popl    %es
        popl    %fs
        popl    %gs

        iret
```

Creating Processes/Tasks

- User-level: fork, vfork, and clone system calls
- In kernel: all map into **do_fork**

```
int do_fork (unsigned long clone_flags,
             unsigned long stack_start,
             struct pt_regs* regs,
             unsigned long stack_size,
             int __user* parent_tidptr,
             int __user* child_tidptr);
```

  - **__user** means not to be dereferenced
  - returns new pid or negative value on error

Creating Processes/Tasks (cont.)

- Parameters
  - clone_flags – control flags, e.g., CLONE_PARENT, which creates a sibling task (like a thread) instead of a child task
  - stack_start – the new task's ESP (in user space; 0 for kernel threads)
  - regs – register values for the new task
  - stack_size – ignored on x86, but needed on some ISAs
  - parent/child_tidptr – filled in as part of system call (**`sys_clone`** only)

Creating Processes/Tasks (cont.)

- **`do_fork`** calls **`copy_process`** (same file) to set up the process descriptor and any other kernel data structures necessary for child execution (e.g., thread_info structure and Kernel Mode stack)
  - only stack_start and regs are used by x86 version

Copy on Write (CoW)

- How do you start a program?
  - at the user level, some other program has to start it
  - usually started by a shell or other interface program
- The mechanism consists of two parts
  - fork, which creates a copy of the current program
  - exec, which loads a new program and starts it

Copy on Write (CoW)

- However, the common case is fork quickly followed by exec

- fork duplicates the process' address space
  - painfully slow in early systems
  - especially when data were then mostly discarded (on exec)

- BSD added vfork (virtual fork)
  - parent blocks while child uses address space
  - after child exits, control of address space returns to parent

Copy on Write (CoW)

- Mach added copy-on-write
  - instead of duplicating data
    - duplicate page tables
    - turn off write permission
  - when either process tries to write to a given page, give it a private copy

- Copy on write is an example of a “lazy” approach to work (opposite is “eager”)
  - allows us to avoid work that won’t actually be useful
  - we’ll see more lazy approaches in the future

Creating Kernel Threads

- What is a kernel thread?
  - a task without an associated address space
  - inherits address space from last user task to execute
  - (all address spaces map the kernel pages and data structures)

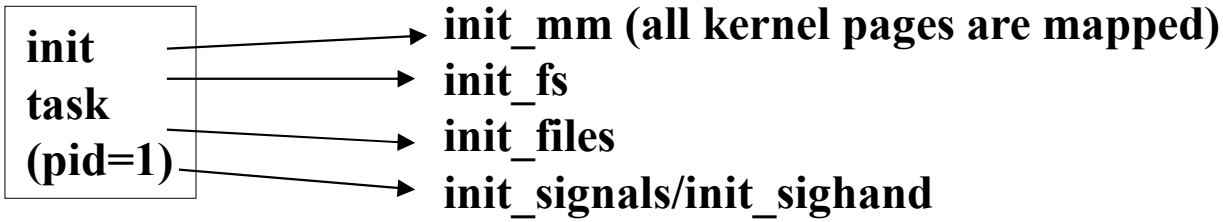
- The interface
  - prototype in asm/processor.h; implementation in arch/i386/kernel/process.c

```
int kernel_thread (int (*fn)(void*), void* arg,
                   unsigned long flags);
```

  - returns new pid or negative value on error

Creating Kernel Threads (cont.)

- init_task is a kernel thread
  - created during boot (in start_kernel); persists until shutdown
  - pid = 1

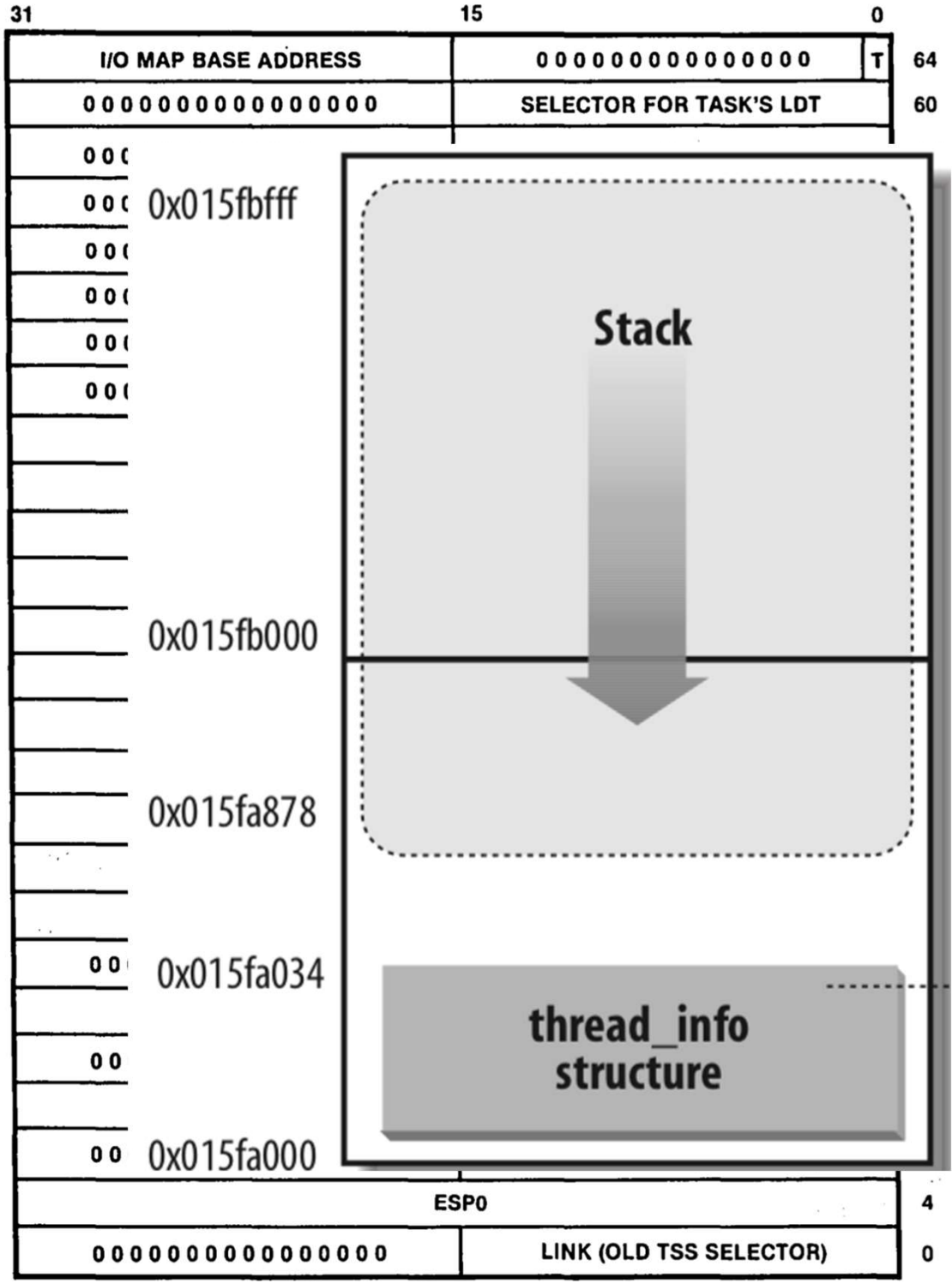


- Other kernel threads
  - ksoftirqd – the soft interrupt daemon (periodically try to execute tasklets)

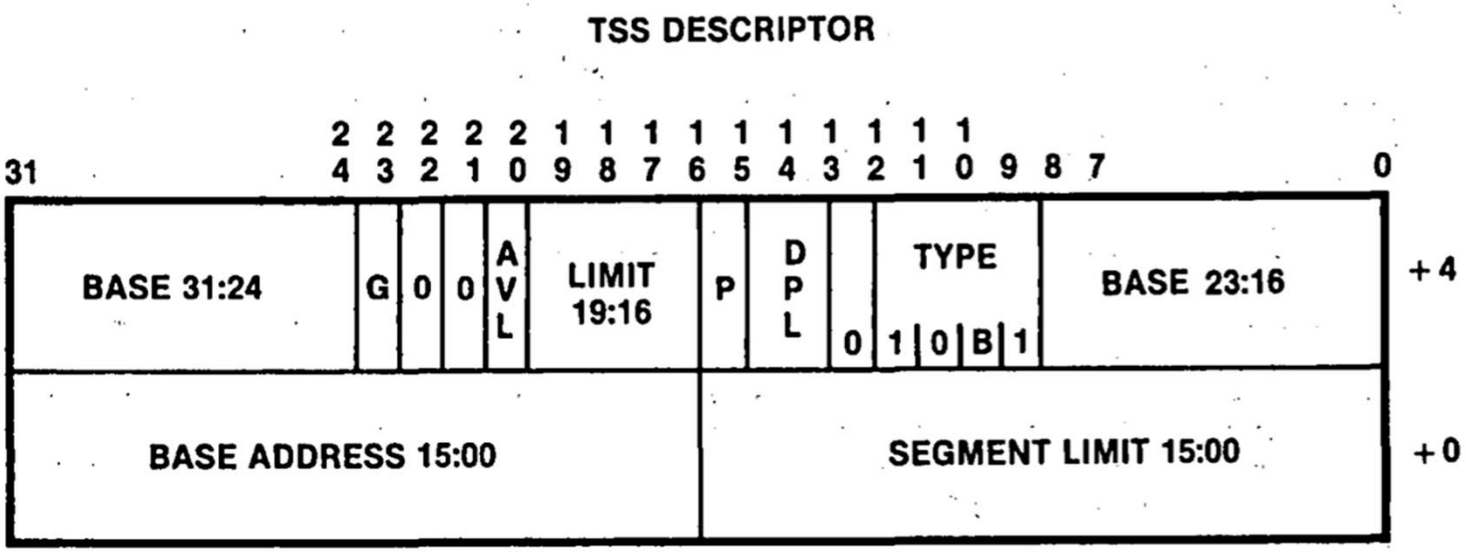
Task State Segment

- Context management is hard, IA-32 is here to help!
- **Task State Segment** (TSS) stores hardware state for a task
- Stack pointers for privileged modes when task is active
- Hardware performs context switch

- Stores hardware state of a task (process)
- Stack pointers for privileged modes
  - Used when handling interrupt
  - ESPi is stack pointer for ring i
- ESP0 pointers to kernel stack
- Selector for task LDT
- PDBR for task page tables
- Linux only uses TSS to specify ESP0 to CPU
- Linux does *not* use TSS for context switching



TSS Descriptor



- TSS is references by descriptor in GDT (cannot be in LDT)
- JMP or CALL to descriptor causes context switch

Scheduling Philosophy

- Several types of jobs exist
  - Interactive
    - examples: editors, GUIs
    - driven by human interaction (e.g., keystrokes, mouse clicks)
    - little or no work to do after each event
    - but important to respond quickly
  - Batch
    - examples: compilation, simulation
    - usually only time to completion matters
    - want a fair share of CPU computation

Scheduling Philosophy (cont.)

- Real-time
  - examples: music, video, teleconferencing
  - periodic deadlines (e.g., 30 frames per second of video)
  - work often only useful if finished on time

- alternative taxonomy: I/O-bound vs. compute-bound

- Goal of scheduling:
  efficient, fair, and responsive (all at once!)

Scheduling Philosophy (cont.)

- General strategy
  - break time into slices
  - allow interactive jobs to preempt the current job based on interrupts
    - typically, a job becomes runnable when an interrupt occurs (e.g., a key is pressed)
    - Linux checks for rescheduling after each interrupt, system call, and exception
  - Linux 2.4 *does not* preempt code running in kernel
  - Linux 2.6 *does* preempt code running in kernel to support real-time applications

Linux General Scheduling Algorithm

- Time broken into epochs
  - each task given a quantum of time (in ticks of 10 milliseconds)
  - run until no runnable task has time left
  - then start a new epoch
- Real-time jobs
  - always given priority over non-real-time jobs
  - prioritized amongst themselves
- Static and dynamic priorities used for non-real-time jobs

Linux General Scheduling Algorithm

- Interactive jobs handled with heuristics
  - heuristic estimates job interactiveness
  - an interactive job
    - can continue to run after running out of time
    - takes turns with other interactive jobs
  - philosophy is that they don't usually use up quantum
  - heuristic ensures that job can't use lots of CPU and still be "interactive"

Task State (fields of task_struct)

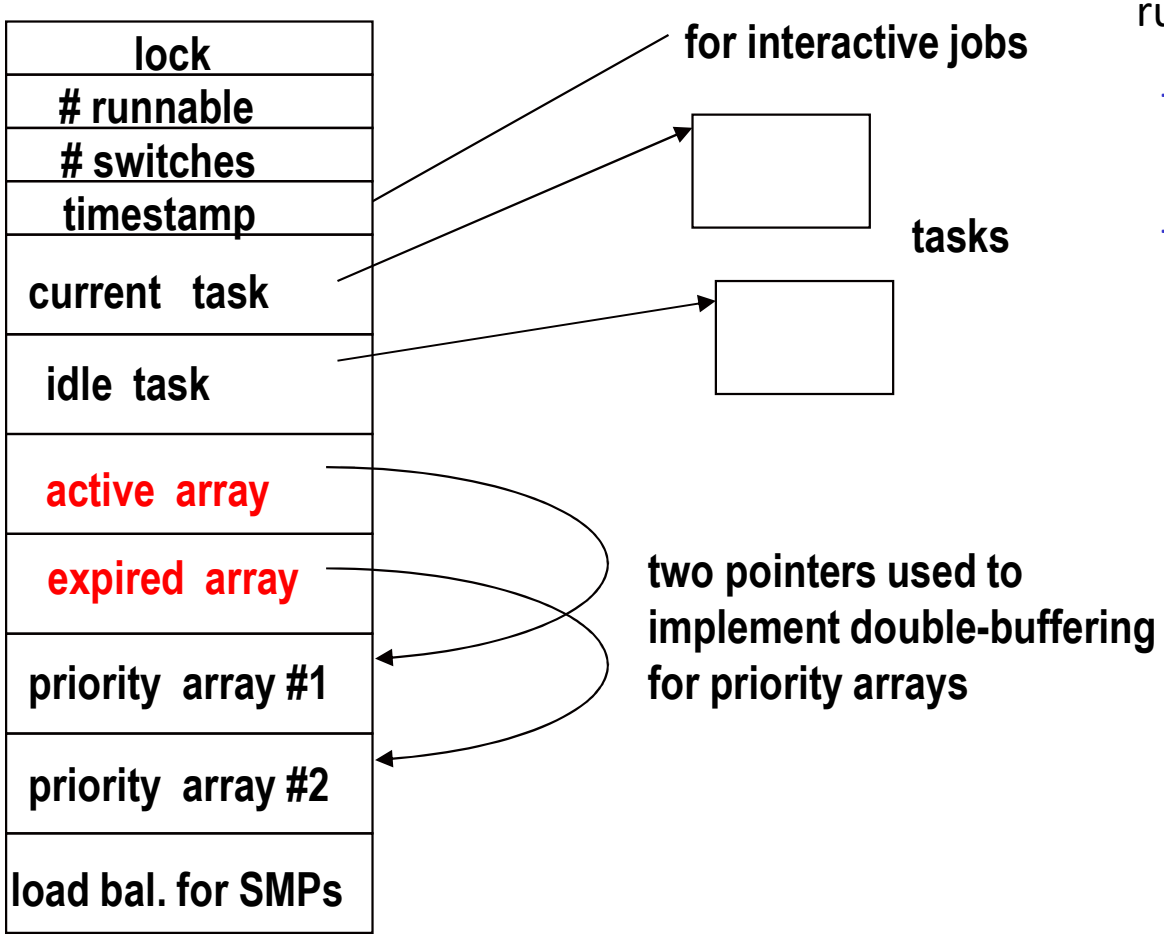
- State field can be one of
  - TASK_RUNNING
    - task is executing currently or waiting to execute
    - task is in a run queue on some processor
  - TASK_INTERRUPTIBLE
    - task is sleeping on a semaphore/condition/signal
    - task is in a wait queue
    - can be made runnable by delivery of signal
  - TASK_UNINTERRUPTIBLE
    - task is busy with something that can't be stopped
    - e.g., a device that will stay in unrecoverable state without further task interaction
    - cannot be made runnable by delivery of signal

Task State (fields of task_struct)

- TASK_STOPPED
  - task is stopped
  - task is not in a queue; must be woken by signal
- TASK_ZOMBIE
  - task has terminated
  - task state retained until parent collects exit status information
  - task is not in a queue

- The swapper process is always runnable (it's an idle loop)

Scheduling Data Structures



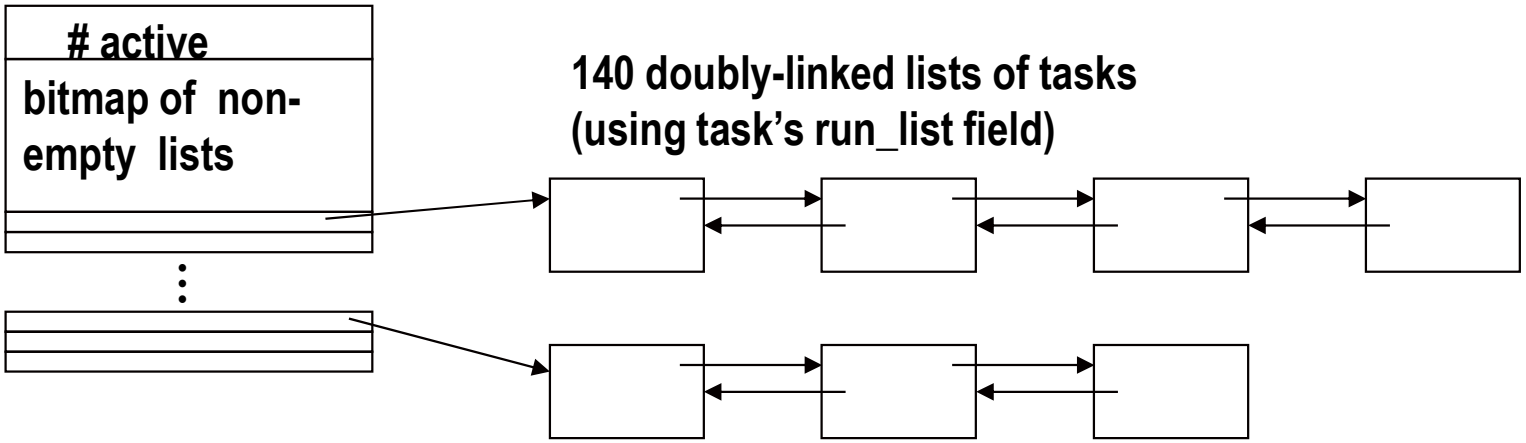
- Each processor has a run queue (struct runqueue in sched.c)
  - each run queue has two priority arrays, i.e., lists of tasks of each priority
  - they are double-buffered to implement epochs

Active processes: runnable processes that have not yet exhausted their time quantum/slice and hence, are allowed to run

Expired processes: runnable processes that have exhausted their time quantum/slice and hence, are not allowed to run until all active processes expire

Priority Array Structure
(struct prio_array in sched.c)

- 100 real-time priorities
- 40 regular priorities
- One list per priority and a bitmap to make finding non-empty list fast



Multiprocessor Systems

- Each CPU has its own runqueue
- A process is placed into exactly only runqueue
  - Process is attached to a particular CPU
- Scheduler periodically rebalances CPUs' runqueues
  - Ensures fair distribution of CPU time (subject to scheduling)

Scheduler Policies: SCHED_FIFO

First-In, First-Out real-time process. When the scheduler assigns the CPU to the process, it leaves the process descriptor in its current position in the runqueue list. If no other higher-priority real time process is runnable, the process will continue to use the CPU as long as it wishes, even if other real-time processes having the same priority are runnable.

- Run until they relinquish the CPU voluntarily
- Priority levels maintained
- Not pre-empted !!

Scheduler Policies: SCHED_RR

Round Robin real-time process. When the scheduler assigns the CPU to the process, it puts the process descriptor at the end of the runqueue list. This policy ensures a fair assignment of CPU time to all SCHED_RR real-time processes that have the same priority.

- Assigned a time slice and run till the time slice is exhausted.
- Once all RR tasks of a given priority level exhaust their time slices, their time slices are refilled and they continue running
- Priority levels are maintained

Scheduler Policies: SCHED_NORMAL

A conventional, time-shared process (used to be called SCHED_OTHER) for normal tasks. Task priority recalculated using a predefined formula. Tasks at the same priority are scheduled using round robin policy.